CS396 – Parallel Computing – Fall 2026
LM2: Threads, Resource Contention, and Deadlock

Whitworth University | Instructor: Qian Mao | Last Modified: 9/22/2026
Individual Assignment

Submission Suggestion:

There are three coding parts in LM2. You could create a separate Visual Studio project for each part, put three subfolders or files in the LM2 folder, and push it back to Whitgit **by Oct. 8**.

Please submit the review report (see part 4) to Canvas **by Oct. 13**.

Part 1: Mutual Exclusion V.1 (20 pts)

There are lots of ways to play with mutual exclusion, but one of the canonical examples is with a bank account.

You are going to develop a program that utilizes a **BankAccount class** with two primary thread safe methods to perform banking operations. The first one will perform deposit; the second will perform withdrawals.

You are going to spawn multiple threads (e.g., 100 threads). Half threads perform deposit, and the other half perform withdrawal. **The deposit amount should be averagely smaller than the withdrawal amount.**

The account value can never go below zero, so if a withdrawal would take more than the account has, it **waits for** a deposit to be made before the withdrawal is made.

Required elements of your **BankAccount** class:
Variables: int accountID; float balance;
Methods: void Withdrawal(float amount), void Deposit(float amount), Print()

Print will show the current value of the account, including accountID. Withdrawal will subtract an amount, blocking until there is enough in the account to withdrawal. Deposit will add value to the account, **regardless of if there is a pending withdrawal**.

Your class and implementation **must be thread safe.** I would suggest utilizing a mutex to protect the critical section of both withdrawing and depositing.

Show that your withdrawal/deposit methods are:

- thread safe **(5pts)**
- efficient **(5pts)**
- correct, i.e., if a withdrawal would take more than the account has, it waits for a deposit to be made before the withdrawal is made. Deposit will add value to the account, regardless of if there is a pending withdrawal. **(5pts)**
- the account balance is correct at the end. **(5pts)**

You can use various method to convince the evaluators, such as analyses, execution result, etc. It might be a good idea to write a report. Note that this part is evaluated by another student in the class.

Hint: The thread initializer really does not like passing variables by reference. You could overcome this by using global variables (not as ideal) or lambda function (more ideal).

Hint: You may find it easier to force the threads to delay a certain amount of time before they perform a deposit / withdrawal. I might suggest taking a look at: http://www.cplusplus.com/reference/thread/this_thread/sleep_for/

Part 2: Mutual Exclusion V.2 (20 pts)

Building off the code started above, you are going to make a new method for your BankAccount. You are going to implement a **PriorityWithdrawal (float)** method. This priority will take precedence over other withdrawals that may be pending, still waiting for the funds to be available.

For this part you should have three types of threads, i.e., Withdrawal, Deposit and Priority Withdrawal. Try to spawn totally 100 threads including all three types.

Show that the deposit / withdrawal/ priority withdrawal is:

- thread safe and efficient **(5pts)**
- the priority order is correct **(5pts)**
- there is no deadlock, and **(5pts)**
- the account balance is correct at the end. **(5pts)**

You can use various method to convince the evaluators, such as analyses, execution result, etc. It might be a good idea to write a report. Note that this part is evaluated by another student in the class.

Part 3: Condition Variables (30 pts)

To demonstrate Condition Variable use, you are going to create two different threaded functions. The first function is going to take an array of int and shuffle them. You are going to need to use a randomizer (e.g., the one we used in LM1). The other function will take that array and sort it. These two functions must coordinate with each other using condition variables. The sorting algorithm will only start to process once the shuffler has completed, and vice versa. Complete 5 cycles of shuffling and sorting. **(20 pts)**

You are going to need to develop a function to check your sort, you can also use this function to check to see if it was randomized. **(10 pts)**

As the problem is stated, you should not need to use thread safe variables for the array part, as the condition variable will protect a race condition on the array. Your solution should be a general solution for arrays of *n* size. You can make this a user input value if you would like.

You can use whatever sorting algorithm you see fit, but you have to implement it yourself.

Note that this part is evaluated by another student in the class.

Part 4: Code Review (30 pts)

Each student will be randomly given another student's code for peer review. Based on the code review, each student will finish a 1-2 page report. You report should address each of the following perspectives and give credits to each question.

- Part 1: Show that your withdrawal/deposit methods are:
  - thread safe **(5pts)**
  - efficient **(5pts)**
  - correct, i.e., if a withdrawal would take more than the account has, it waits for a deposit to be made before the withdrawal is made. Deposit will add value to the account, regardless of if there is a pending withdrawal. **(5pts)**

    - the account balance is correct at the end. **(5pts)**
- Part 2: Show that the deposit / withdrawal/ priority withdrawal is:
    - thread safe and efficient **(5pts)**
    - the priority order is correct **(5pts)**
    - there is no deadlock, and **(5pts)**
    - the account balance is correct at the end. **(5pts)**
- Part 3:
    - Complete 5 cycles of shuffling and sorting in turn. **(20 pts)**
    - A function (or functions) that checks the correctness of sorting and randomizing. **(10 pts)**